

Engenharia de Segurança

Engenharia de Qualidade e Confiabilidade

Prof. Me. Sergio Souza Novak — UniRV

6 de abril de 2026



1) Caso Real: Aeroporto de Varsóvia

Em **1993**, um Airbus colidiu durante o pouso no aeroporto de Varsóvia:

- **2 mortes e 54 feridos.**
- Causa significativa: **falha no software de controle.**
- O software **reduziu a eficiência do sistema de frenagem.**
- As condições da pista não foram corretamente interpretadas.



“Ilustra que o software é um componente fundamental em sistemas críticos para preservar a vida humana.”

2) O que é Segurança (*Safety*)?

Definição

Um sistema é considerado **seguro** se operar sem **falhas catastróficas** — falhas que causem ou possam causar morte ou lesão às pessoas.

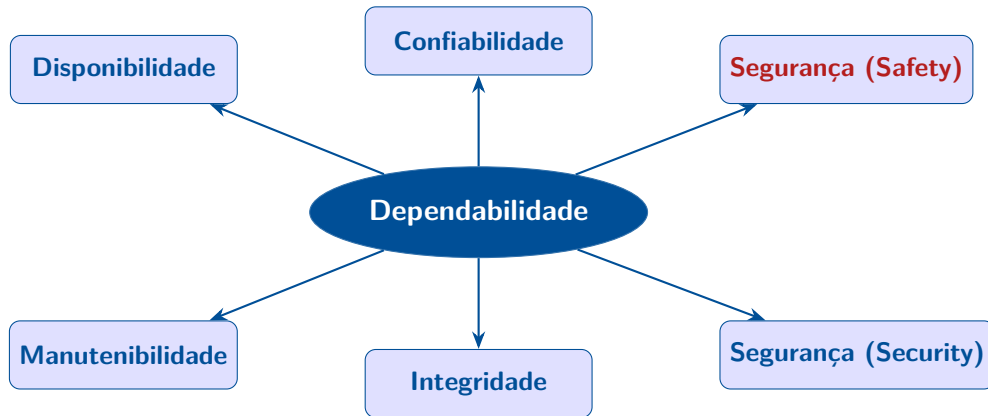
- Sistemas cujas falhas causem **danos ambientais** também são críticos em segurança.
- Danos ambientais (ex.: vazamento químico) podem levar a subsequentes mortes ou lesões.
- Segurança (*safety*) é uma das principais propriedades da **dependabilidade**.

Atenção

Safety $\neq$ *Security*. **Safety** trata de falhas que causam danos físicos; **Security** trata de proteção contra ataques e acesso não autorizado.



3) Segurança e Dependabilidade



4) O Duplo Papel do Software na Segurança

O software em sistemas críticos em segurança exerce **dois papéis**:

Papel 1: Controle Direto

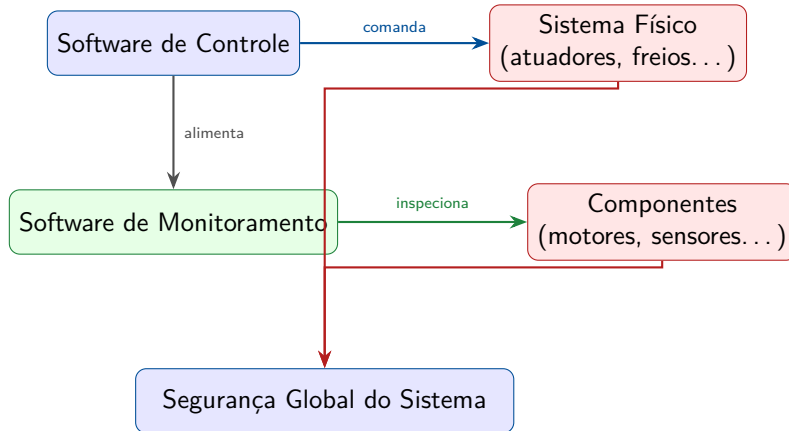
- O sistema é **controlado por software**.
- Decisões do software $\Rightarrow$ ações críticas.
- O comportamento do software está **diretamente ligado** à segurança global.

Papel 2: Monitoramento

- Software **confere e monitora** outros componentes.
- Ex.: monitorar todos os componentes do motor de uma aeronave.
- Falha do software de monitoramento $\Rightarrow$ outros componentes podem falhar.



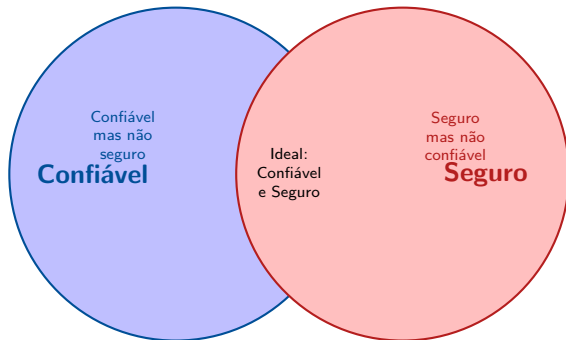
5) Fluxo: Controle e Monitoramento



6) Confiabilidade $\neq$ Segurança

Armadilha Comum

Um sistema **confiável** pode **não ser seguro**, e vice-versa!



O acidente em Varsóvia: o sistema era tecnicamente confiável, porém falhou em segurança.



7) Por que Sistemas Confiáveis Podem Não Ser Seguros?

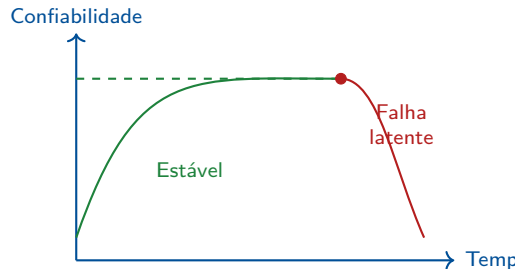
Quatro razões principais:

- ① **Defeitos não detectados** podem ficar inativos por anos e causar falhas inesperadas.
- ② **Especificações incompletas** não descrevem o comportamento exigido em situações críticas.
- ③ **Mau funcionamento de hardware** gera sinais fora dos intervalos tratáveis pelo software.
- ④ **Entradas inesperadas dos operadores** individualmente corretas, mas perigosas em combinação.



8) Razão 1: Defeitos Inativos (*Dormant Bugs*)

- Nunca podemos ter **100% de certeza** de que um software é livre de defeitos.
- Defeitos podem **ficar inativos por longo tempo**.
- Falhas de software ocorrem **após anos** de operação confiável.
- Ambientes de produção revelam combinações de estados impossíveis de testar exaustivamente.



9) Razão 2: Especificações Incompletas

Estudo de Lutz (1993) — Sistemas Embarcados

“As dificuldades com os requisitos são a causa principal dos erros de software relacionados à segurança, que persistiram até a integração e teste do sistema.”

- Alta porcentagem de mau funcionamento resulta de **erros de especificação**.
- Trabalho de Veras et al. (2010) em sistemas espaciais confirma: **erros de requisitos** ainda são um problema importante.
- Situações críticas raramente testadas em especificação $\Rightarrow$ **comportamento indefinido**.



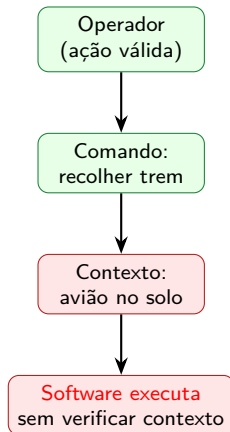
10) Razão 3: Mau Funcionamento de Hardware

- Sensores e atuadores podem se comportar de modo **imprevisível** quando próximos da falha física.
- Geram **sinais fora dos intervalos** tratáveis pelo software.
- O software pode **falhar** ou **interpretar equivocadamente** esses sinais.



11) Razão 4: Entradas Inesperadas dos Operadores

- Operadores podem gerar entradas **individualmente corretas**, mas perigosas em combinação.
- **Exemplo real:** Técnico pressionou botão de recolhimento do trem de pouso enquanto aeronave estava no **solo**.
- O software executou a instrução corretamente.
- O sistema **deveria ter desabilitado** o comando a menos que o avião estivesse no ar.



12) Engenharia de Sistemas Críticos em Segurança

- Segurança deve ser considerada **tanto quanto a confiabilidade** durante o desenvolvimento.
- É necessário desenvolver uma **compreensão das situações** que podem levar a falhas de segurança.
- O software deve ser projetado para que essas falhas **não ocorram**.
- Técnicas de engenharia da confiabilidade (Capítulo 11) são aplicáveis, mas **não suficientes** sozinhas.

Foco

Métodos e técnicas específicos foram desenvolvidos para a **engenharia de software crítico em segurança**.

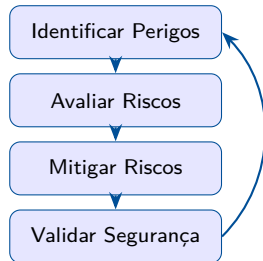
13) Análise de Perigos (*Hazard Analysis*)

Objetivo

Identificar condições de operação que podem levar a acidentes ou incidentes.

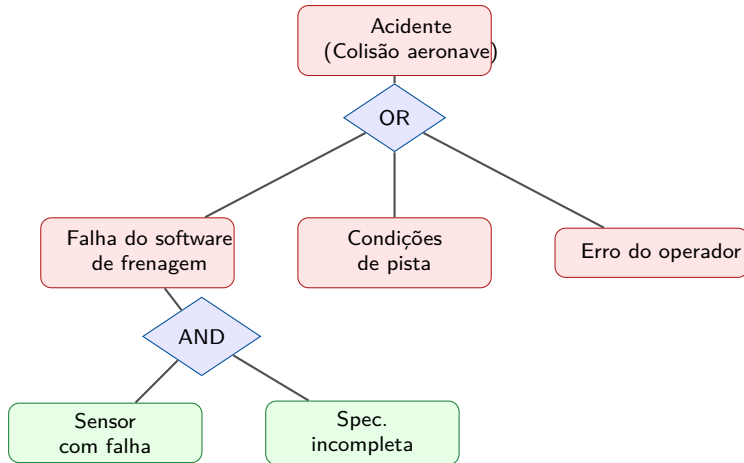
Perguntas-chave:

- Quais **estados do sistema** podem causar dano?
- Quais **seqüências de eventos** levam a esses estados?
- Como **prevenir** ou **mitigar** esses estados?



14) Árvore de Falhas (*Fault Tree Analysis* — *FTA*)

Técnica clássica para identificar **causas raízes** de falhas críticas (top-down).



15) FMEA — Análise de Modos de Falha e Efeitos

Definição

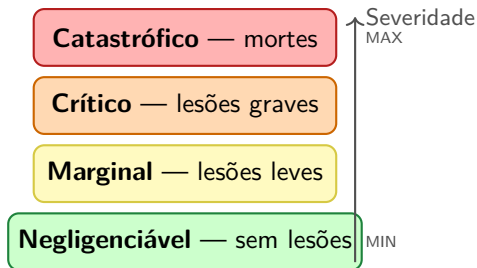
FMEA (*Failure Mode and Effects Analysis*): método sistemático bottom-up para identificar como componentes podem falhar e quais os efeitos dessas falhas no sistema.

Componente	Modo de Falha	Efeito	Severidade
Sensor de velocidade	Leitura incorreta	Frenagem inadequada	Crítico
Software de controle	Bug latente	Comando errado	Crítico
Atuador de freio	Resposta lenta	Distância maior	Alto
Interface operador	Comando inválido	Ação indesejada	Moderado



16) Categorias de Criticidade em Segurança

Sistemas críticos são classificados por **nível de risco**:



17) Requisitos de Segurança

- Derivados a partir da **análise de perigos**.
- Devem cobrir:
 - **Prevenção** de estados perigosos.
 - **Deteção** antecipada de falhas.
 - **Contenção** e limitação de danos.
 - **Recuperação** após falha.
- Estudos mostram que **erros de requisitos** são a causa mais frequente de falhas de segurança.

Boa prática

Toda situação crítica identificada na análise de perigos deve ter um **requisito de segurança correspondente** na especificação.

18) Técnicas de Programação para Sistemas Seguros

Prevenção de Falhas:

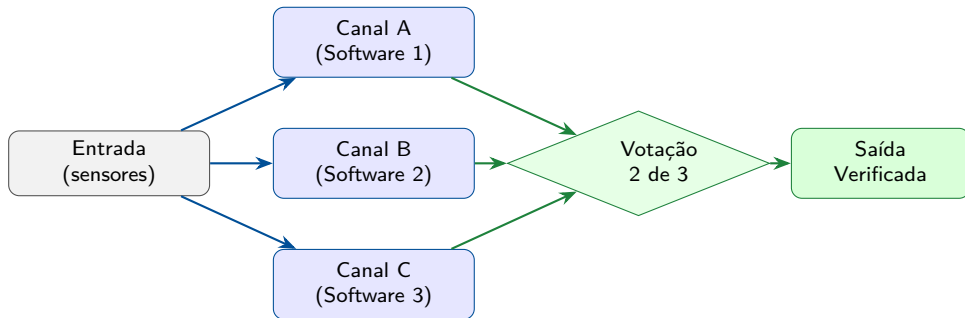
- Uso de linguagens fortemente tipadas.
- Proibição de construções inseguras (ponteiros soltos, etc.).
- Programação defensiva com *assertions*.
- Verificação formal de propriedades críticas.

Tolerância a Falhas:

- **Redundância**: múltiplos canais independentes.
- **Votação** entre sistemas redundantes.
- **Estado seguro**: falha leva a estado pré-definido e seguro.
- **Watchdog timers**: reinicialização automática.



19) Arquitetura com Redundância — Diagrama



Se um canal produzir resultado diferente, os outros dois vencem na votação $\Rightarrow$ falha tolerada.

20) Validação e Verificação em Sistemas Seguros

Verificação

Estamos construindo o sistema **corretamente**? (conformidade com a especificação)

Validação

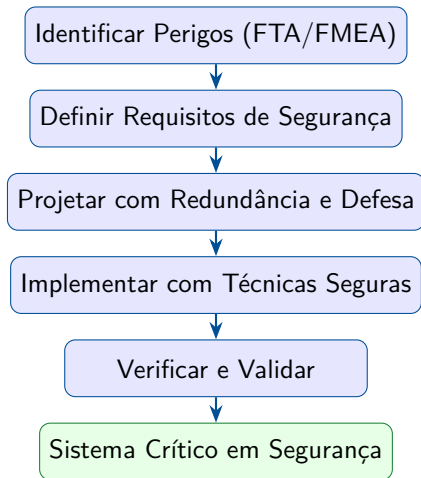
Estamos construindo o sistema **correto**? (atende às necessidades reais de segurança)

Técnicas aplicadas:

- **Revisões formais** e inspeções de código.
- **Testes de segurança**: cenários adversariais e injeção de falhas.
- **Verificação formal** de propriedades críticas (model checking).
- **Certificação** por organismos independentes (ex.: DO-178C para aviação).



21) Resumo



22) Sistemas Críticos em Segurança

Definição

São aqueles em que é essencial que a operação seja **always safe**, nunca provocando danos às pessoas ou ao meio ambiente, independentemente de estar em conformidade com a especificação.

Exemplos incluem:

- Sistemas de controle e monitoramento em **aeronaves**.
- Controle de processos nas indústrias **química e farmacêutica**.
- Sistemas de controle em **automóveis**.



23) Software Crítico em Segurança: Primário

- **Software Embarcado:** Atua como controlador de um sistema físico.
- **Falha Direta:** O mau funcionamento pode provocar falha do hardware, resultando em lesões ou prejuízo ambiental.
- **Exemplo:** Software da **bomba de insulina**.

Complexidade

Essencial para gerenciar grandes quantidades de sensores e atuadores (ex: aeronaves militares aerodinamicamente instáveis, que requerem ajuste contínuo).



24) Software Crítico em Segurança: Secundário

- **Dano Indireto:** O software pode causar uma lesão indiretamente através de uma falha em outros processos.
- **Exemplos:**
 - **CAD (Projeto Assistido por Computador):** Defeito no projeto de um objeto pode levar a falha física posterior.
 - **Mentcare:** Falha no sistema de saúde mental pode levar a tratamento incorreto e lesão ao paciente ou terceiros.
- **Infraestrutura Nacional:** Interrupção prolongada (eletricidade, esgoto) pode levar a doenças infecciosas e morte.



25) Abordagens para Segurança

A engenharia de segurança utiliza técnicas dirigidas por perigos:

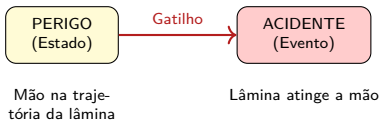
- 1 **Prevenção de perigos:** Sistema projetado para evitar o perigo.
Ex: Cortadora de papel que exige as duas mãos simultaneamente.
- 2 **Deteccção e remoção de perigos:** Detectar antes que o acidente ocorra.
Ex: Válvula de alívio em indústria química para evitar explosão.
- 3 **Limitação dos danos:** Minimizar danos após o acidente.
Ex: Extintores de incêndio automáticos em motores de avião.

26) O que é um Perigo (*Hazard*)?

Conceito

Um **perigo** é um **estado do sistema** que poderia levar a um acidente.

- **Perigo $\neq$ Acidente:** Situações perigosas ocorrem frequentemente sem resultar em problemas.
- Contudo, **acidentes são sempre precedidos por perigos.**
- **Reduzir a quantidade de perigos é a chave para reduzir acidentes.**



27) Terminologia de Segurança

Termo	Definição
Acidente	Evento não planejado que resulta em lesão, morte ou dano ambiental.
Dano	Medida da perda resultante de um acidente (de lesão leve a morte).
Perigo	Condição com potencial para causar ou contribuir para um acidente.
Probabilidade	Chance de ocorrência de eventos que criam um perigo (ex: 1/100).
Gravidade	Avaliação do pior dano possível (ex: catastrófica, pequena).
Risco	Medida da probabilidade do sistema causar um acidente.

28) "Acidentes Normais"(Perrow, 1984)

- **Falhas Combinadas:** Quase todos os acidentes sérios ocorrem por uma combinação de falhas em diferentes partes do sistema.
- **Interações não previstas:** Sistemas complexos dificultam a previsão de como falhas de subsistemas interagem.
- **Inevitabilidade:** Em sistemas muito complexos, acidentes devem ser considerados inevitáveis ("Normais").

Consequência

A falha de um item (ex: ar condicionado) pode levar ao superaquecimento de outro, gerando sinais errados que confundem o software.

29) Vantagens do Controle por Software

Apesar da complexidade, o software contribui para a segurança:

- **Monitoramento superior:** Consegue monitorar uma gama maior de condições que sistemas eletromecânicos.
- **Travas de segurança sofisticadas:** Permite a implementação de estratégias de controle complexas.
- **Redução de exposição humana:** Diminui o tempo que pessoas precisam passar em ambientes perigosos.
- **Flexibilidade:** Pode ser adaptado com facilidade e usa hardware leve e confiável.

Impossível fazer um sistema 100% seguro; a sociedade decide se os benefícios valem o risco.



30) Requisitos de Segurança — Lições de Varsóvia

- **O Software funcionou, mas o Sistema falhou:** Em Varsóvia, não havia bugs no código; o erro estava na **especificação incompleta**.
- **Conclusão:** A segurança não depende apenas de boa engenharia de software, mas de requisitos que prevejam situações raras.
- **Requisitos de Segurança:** São requisitos funcionais que definem recursos de verificação, recuperação e proteção contra falhas e ataques.



31) Requisitos do Tipo "Não Deve" (*Shall Not*)

- **Diferença Fundamental:**

- *Requisitos funcionais normais*: Definem o que o sistema **deve fazer**.
- *Requisitos de segurança*: Definem o comportamento **inaceitável** (o que o sistema **não deve fazer**).

- **Implementação**: Não podem ser implementados diretamente; devem ser decompostos em requisitos funcionais específicos ou decisões de projeto (hardware).

- **Equilíbrio**: É preciso encontrar um balanço aceitável entre **segurança e funcionalidade** para evitar custos excessivos.



32) Exemplos de Requisitos "Não Deve"

Aviação

"O sistema **não deve** permitir o acionamento do impulso reverso quando a aeronave estiver em voo."

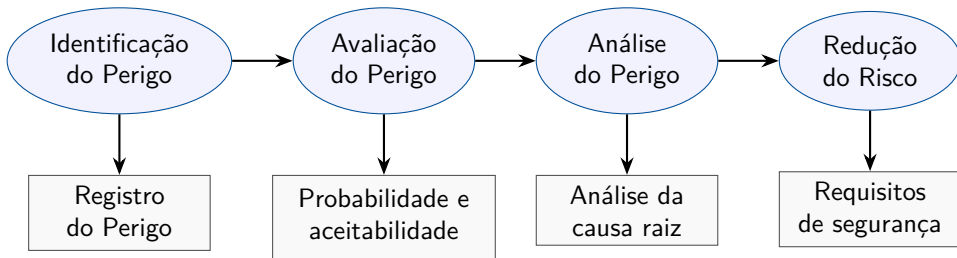
Sistemas de Alarme

"O sistema **não deve** permitir a ativação simultânea de mais de três sinais de alarme."

Automotivo

"O sistema de navegação **não deve** permitir que os usuários definam o destino quando o carro estiver em movimento."

33) Processo de Especificação Dirigida por Perigos



34) 12.2.1 Identificação do Perigo

- Começa pela determinação das **classes de perigos**: físico, elétrico, biológico, radiação, falha de serviço, etc.
- **Técnicas**: Brainstorming com especialistas de domínio, engenheiros e conselheiros de segurança.
- *Exemplo Bomba de Insulina*: Envolve médicos, físicos médicos, engenheiros e projetistas de software.

Objetivo

Identificar perigos individuais e possíveis combinações nocivas.



35) Exemplo: Perigos na Bomba de Insulina

Falhas de Serviço:

- Overdose de insulina.
- Dosagem insuficiente.
- Falha no monitoramento.

Elétricos:

- Esgotamento da bateria.
- Interferência com marca-passo.

Físicos:

- Máquina quebrar no corpo.
- Montagem incorreta.

Biológicos:

- Infecção ou reação alérgica.

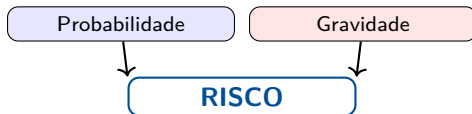
36) Registro de Perigos (*Hazard Registry*)

- **Documento Formal:** Registra decisões, análises e avaliações de segurança.
- **Importância Legal:**
 - Prova que os engenheiros prestaram o cuidado devido e atenção aos perigos previsíveis.
 - Utilizado em inquéritos para mostrar que não houve **negligência**.
- **Uso Contínuo:** Base para o caso de segurança submetido aos reguladores.



37) 12.2.2 Avaliação do Perigo

- Foca na compreensão dos fatores que levam ao perigo e suas **consequências**.
- Decide como gerenciar o risco associado.
- **Declaração de Aceitabilidade:** Expressa o risco combinando probabilidade e gravidade.

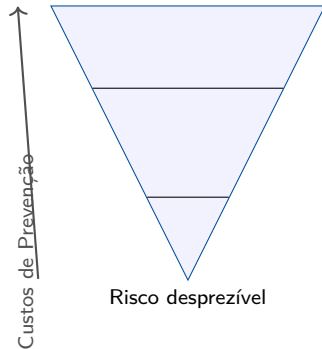


38) Categorias de Risco

- 1 **Riscos Intoleráveis:** Ameaçam a vida humana. O sistema **não deve** permitir que surjam ou deve detectá-los antes do acidente.
- 2 **Riscos ALARP (*As Low As Reasonably Practical*):** Consequências menos graves ou baixa probabilidade. Minimizados sujeitos ao custo.
- 3 **Riscos Aceitáveis:** Danos pequenos. Reduzidos se não aumentarem custo ou prazo significativamente.



39) Triângulo do Risco



Região Inaceitável: Risco não tolerado

Região ALARP: Risco tolerado se prevenção for inviável

Região Aceitável

40) Classificação de Riscos (Bomba de Insulina)

Perigo Identificado	Prob.	Grav.	Risco	Aceitabilidade
1. Overdose de insulina	Média	Alta	Alto	Intolerável
2. Dose insuficiente	Média	Baixa	Baixo	Aceitável
3. Falha de monitoramento	Média	Média	Baixo	ALARP
4. Falha de energia	Alta	Baixa	Baixo	Aceitável
5. Encaixe incorreto	Alta	Alta	Alto	Intolerável
6. Quebra no paciente	Baixa	Alta	Médio	ALARP
10. Reação alérgica	Baixa	Baixa	Baixo	Aceitável

A aceitabilidade depende da tolerância da

sociedade e custos de prevenção.

41) O Papel do Software no Monitoramento

- Software monitora perigos de hardware (ex: bateria, encaixe da máquina).
- **Aviso Prévio:** Permite que o perigo seja detectado antes do acidente.
- **Estimativa de Risco:** É difícil e usa termos qualitativos (*provável, raro, alto*) por falta de dados estatísticos em acidentes raros.
- **Crítico para a Segurança:** Falha no sistema de monitoramento pode ter consequências sérias se ignorar uma falha de hardware.



42) 12.2.3 Análise do Perigo

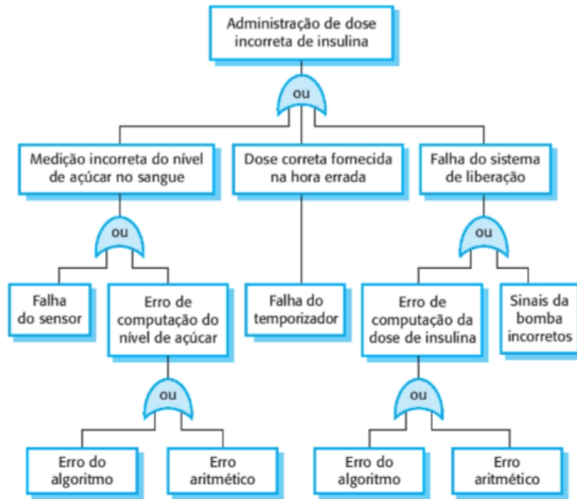
- **Objetivo:** Descobrir as causas principais e combinações de eventos que resultam em perigo.
- **Abordagens:**
 - **Top-down (Dedutiva):** Começa pelo perigo e chega até a falha do sistema. (Mais fácil de usar).
 - **Bottom-up (Indutiva):** Começa pela falha do sistema e identifica quais perigos resultam dela.
- **Destaque:** A técnica mais utilizada é a **Análise da Árvore de Defeitos (FTA)**.



43) Análise da Árvore de Defeitos (FTA)

- Técnica top-down desenvolvida originalmente para hardware, mas amplamente aplicada ao software.
- **Processo:**
 - 1 Coloca-se o **perigo na raiz** da árvore.
 - 2 Identifica-se os estados do sistema que levam a esse perigo.
 - 3 Continua-se a decomposição até chegar à **causa raiz**.
- **Princípio:** Perigos que surgem apenas de uma combinação de causas são menos propensos a causar acidentes do que aqueles com causa única.

44) Árvore de Defeitos — Bomba de Insulina



45) Análise dos Ramos da Árvore

- **Ramo Esquerdo (Medição):** A falha pode vir do sensor físico ou de erros de computação (algoritmo ou ponto flutuante).
- **Ramo Central (Sincronismo):** Problemas de fornecer a dose cedo ou tarde demais resultam de falhas no temporizador.
- **Ramo Direito (Liberação):** Examina falhas no envio dos sinais corretos para a bomba ou erros no cálculo final da dose.

Hardware vs. Software

Árvores de hardware podem sugerir requisitos de software para diagnóstico e correção (ex: autoverificação 4-6 vezes por hora).

46) 12.2.4 Redução do Risco

Três estratégias principais (geralmente combinadas):

- ❶ **Proteção contra Perigos:** O sistema é projetado para que o perigo simplesmente não ocorra.
- ❷ **Deteção e Remoção:** Perigos são identificados e neutralizados antes do acidente.
- ❸ **Limitação de Danos:** Minimizar as consequências caso o acidente ocorra (ex: seguros, contenção física).

Estado Seguro

Para a bomba de insulina, o **estado seguro** é o desligamento (injeção zero), pois no curto prazo isso não ameaça a saúde.

47) Tratamento de Erros e Estado Seguro

Soluções para falhas de software na Bomba de Insulina:

- **Erro Aritmético:**

- Incluir tratamento de exceções para todos os erros possíveis.
- Ação padrão: **desligar o sistema** e ativar alarme.

- **Erro Algorítmico:**

- Comparar a dose calculada com a dose anterior (se for muito maior, pode ser erro).
- Monitorar a sequência de doses e limitar dosagens futuras excessivas.



48) Exemplos de Requisitos de Segurança

- **RS1:** O sistema não deve fornecer nenhuma dose única maior que a máxima especificada.
- **RS2:** O sistema não deve fornecer uma dose cumulativa diária maior que a máxima.
- **RS3:** O sistema deve incluir diagnóstico de hardware executado pelo menos 4 vezes/hora.
- **RS4:** O sistema deve incluir tratamento para todas as exceções identificadas.
- **RS5:** Um alarme audível deve soar em qualquer anomalia de HW ou SW.
- **RS6:** Em caso de alarme, o fornecimento de insulina deve ser suspenso até o reset manual.



Obrigado!

Dúvidas?

